



数据结构

Data Structure

许 蕾

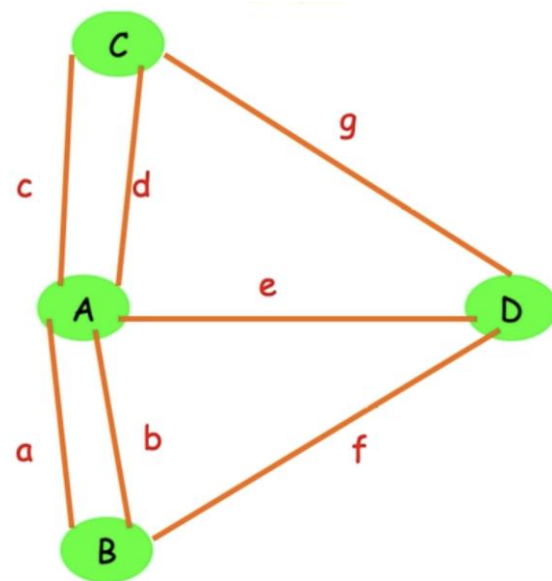
xlei@nju.edu.cn



第八章 图



- 图的基本概念
- 图的存储结构
- 图的遍历与连通性
- 最小生成树
- 最短路径
- 活动网络





8.1 图的基本概念



- **图定义** 图是由顶点(vertex)集合以及顶点之间的关系集合组成的一种数据结构:

$$G = (V, E)$$

其中 $V = \{x \mid x \in \text{某个数据对象}\}$
是顶点的有穷非空集合;

$$E = \{(x, y) \mid x, y \in V\}$$

或 $E = \{<x, y> \mid x, y \in V \ \&\& \ Path(x, y)\}$
是顶点之间关系的有穷集合, 也叫做边(edge)集合。

- $Path(x, y)$ 表示从 x 到 y 的一条单向通路, 它是有方向的。

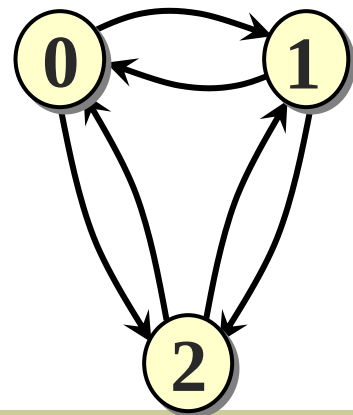
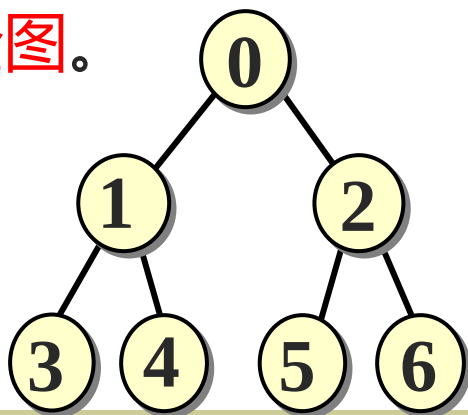
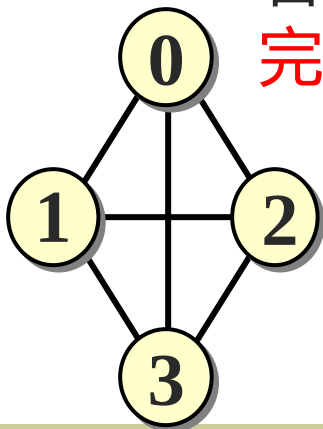


- **有向图与无向图**

- 在**有向图**中, 顶点对 $\langle x, y \rangle$ 是有序的。
- 在**无向图**中, 顶点对 (x, y) 是无序的。

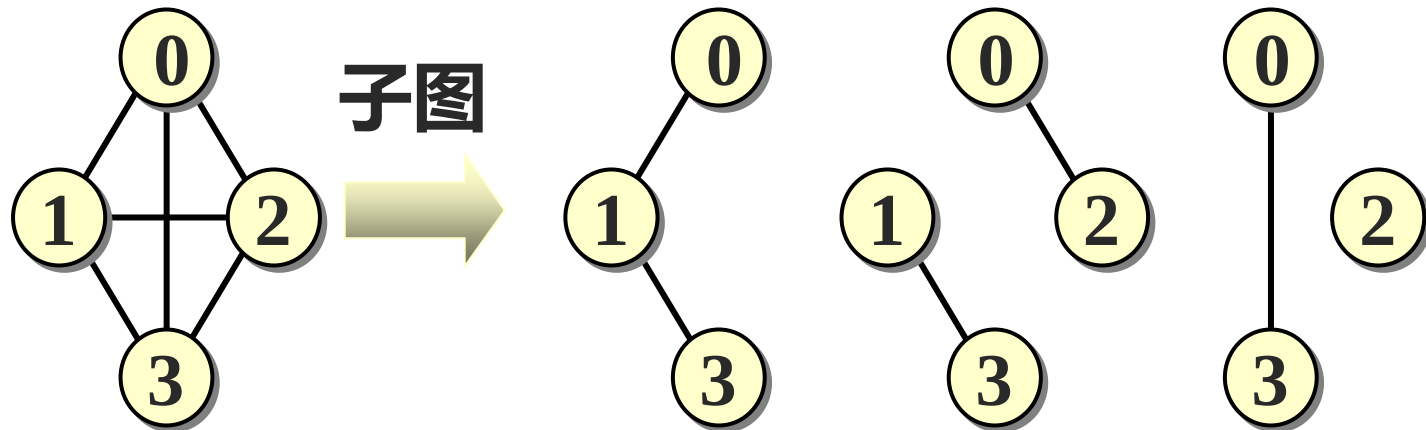
- **完全图**

- 若有 n 个顶点的无向图有 $n(n-1)/2$ 条边, 则此图为**无向完全图**。
- 若有 n 个顶点的有向图有 $n(n-1)$ 条边, 则此图为**有向完全图**。





- **邻接顶点** 如果 (u, v) 是 $E(G)$ 中的一条边, 则称 u 与 v 互为邻接顶点。
- **子图** 设有两个图 $G = (V, E)$ 和 $G' = (V', E')$ 。若 $V' \subseteq V$ 且 $E' \subseteq E$, 则称图 G' 是图 G 的子图。



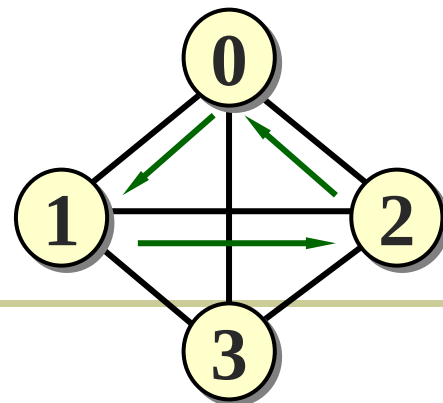
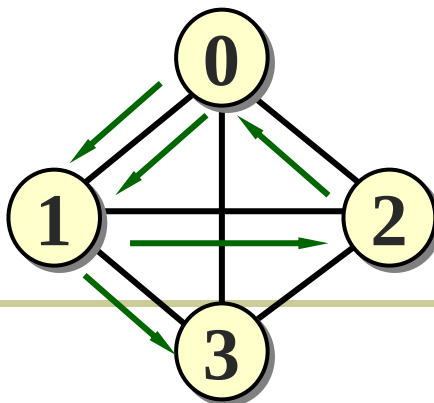
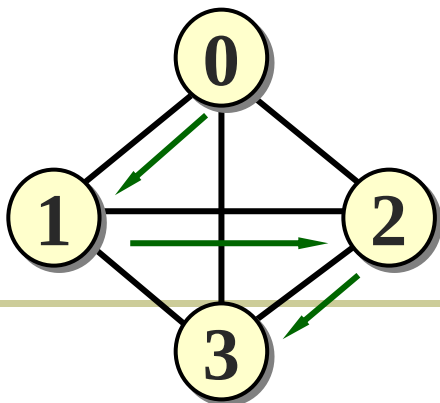
- **权或权重(weight)** 在某些图中, 边具有与它相关的数值, 称为权重。
- **带权图**也叫作网络(network)。



- **顶点的度 (degree)** 一个顶点 v 的度是与它相关联的边的条数, 记作 $\deg(v)$ 。
- 在有向图中, 顶点 v 的**入度**是以 v 为终点的有向边的条数, 记作 $\text{indeg}(v)$; 顶点 v 的**出度**是以 v 为始点的有向边的条数, 记作 $\text{outdeg}(v)$ 。
- 在有向图中, 顶点的度等于该顶点的**入度**与**出度**之和。
- **路径** 在图 $G = (V, E)$ 中, 若从顶点 v_i 出发, 沿一些边经过若干顶点 $v_{p1}, v_{p2}, \dots, v_{pm}$, 到达顶点 v_j , 则称顶点序列 $(v_i, v_{p1}, v_{p2}, \dots, v_{pm}, v_j)$ 为从顶点 v_i 到顶点 v_j 的一条路径。它经过的边 (v_i, v_{p1}) 、 (v_{p1}, v_{p2}) 、 $\dots$ 、 (v_{pm}, v_j) 都是来自于 E 的边。



- **路径长度**
 - 非带权图的路径长度是指此路径上边的条数。
 - 带权图的路径长度是指路径上各边的权值之和。
- **简单路径** 若路径上各顶点 $v_1, v_2, \dots, v_m$ 均不互相重复, 则称这样的路径为简单路径。
- **回路** 若路径上第一个顶点 v_1 与最后一个顶点 v_m 重合, 则称这样的路径为回路或环。





- **连通图与连通分量** 在**无向图**中, 若从顶点 v_1 到顶点 v_2 有路径, 则称顶点 v_1 与 v_2 是连通的。如果图中任意一对顶点都是连通的, 则称此图是连通图。非连通图的极大连通子图叫做**连通分量**。
- **强连通图与强连通分量** 在**有向图**中, 若对于每一对顶点 v_i 和 v_j , 都存在一条从 v_i 到 v_j 和从 v_j 到 v_i 的路径, 则称此图是强连通图。非强连通图的极大强连通子图叫做**强连通分量**。
- **生成树(spanning tree)** 一个无向连通图的生成树是其极小连通子图, 在 n 个顶点的情形下, 有 $n-1$ 条边。若是有向图, 则可能得到它的由若干有向树组成的生成森林。



图的抽象数据类型



```
class Graph {
```

```
//对象: 由一个非空顶点集合和一个边集合构成
```

```
//每条边由一个顶点对来表示。
```

```
public:
```

```
    Graph(); //建立一个空的图
```

```
    void insertVertex (const T& vertex);
```

```
        //插入一个顶点vertex, 该顶点暂时没有入边
```

```
    void insertEdge (int v1, int v2, int weight);
```

```
        //在图中插入一条边(v1, v2, w)
```

```
    void removeVertex (int v);
```

```
        //在图中删除顶点v和所有关联到它的边
```



```
void removeEdge (int v1, int v2);  
    //在图中删去边(v1,v2)  
bool IsEmpty();  
    //若图中没有顶点, 则返回true, 否则返回false  
T getWeight (int v1, int v2);  
    //函数返回边 (v1,v2) 的权值  
int getFirstNeighbor (int v);  
    //给出顶点 v 第一个邻接顶点的位置  
int getNextNeighbor (int v, int w);  
    //给出顶点 v 的某邻接顶点 w 的下一个邻接顶点  
};
```



- **Adjacent(G, x, y)**: 判断图 G 是否存在边 $\langle x, y \rangle$ 或 (x, y) 。
- **Neighbors(G, x)**: 列出图 G 中与结点 x 邻接的边。
- **InsertVertex(G, x)**: 在图 G 中插入顶点 x 。
- **DeleteVertex(G, x)**: 从图 G 中删除顶点 x 。
- **AddEdge(G, x, y)**: 若无向边 (x, y) 或有向边 $\langle x, y \rangle$ 不存在, 则向图 G 中添加该边。
- **RemoveEdge(G, x, y)**: 若无向边 (x, y) 或有向边 $\langle x, y \rangle$ 存在, 则从图 G 中删除该边。
- **FirstNeighbor(G, x)**: 求图 G 中顶点 x 的第一个邻接点, 若有则返回顶点号。若 x 没有邻接点或图中不存在 x , 则返回-1。
- **NextNeighbor(G, x, y)**: 假设图 G 中顶点 y 是顶点 x 的一个邻接点, 返回除 y 之外顶点 x 的下一个邻接点的顶点号, 若 y 是 x 的最后一个邻接点, 则返回-1。
- **Get_edge_value(G, x, y)**: 获取图 G 中边 (x, y) 或 $\langle x, y \rangle$ 对应的权值。
- **Set_edge_value(G, x, y, v)**: 设置图 G 中边 (x, y) 或 $\langle x, y \rangle$ 对应的权值为 v 。

此外, 还有**图的遍历算法**, 包括深度优先遍历和广度优先遍历。



8.2 图的存储结构



- **图的模板基类** 在模板类定义中的数据类型参数表 `<class T, class E>` 中, T是顶点数据的类型, E是边上所附数据的类型。
- 这个模板基类是按照最复杂的情况 (即带权无向图) 来定义的, 如果需要使用非带权图, 可将数据类型参数表 `<class T, class E>` 改为 `<class T>`。
- 如果使用的是有向图, 也可以对程序做相应的改动。

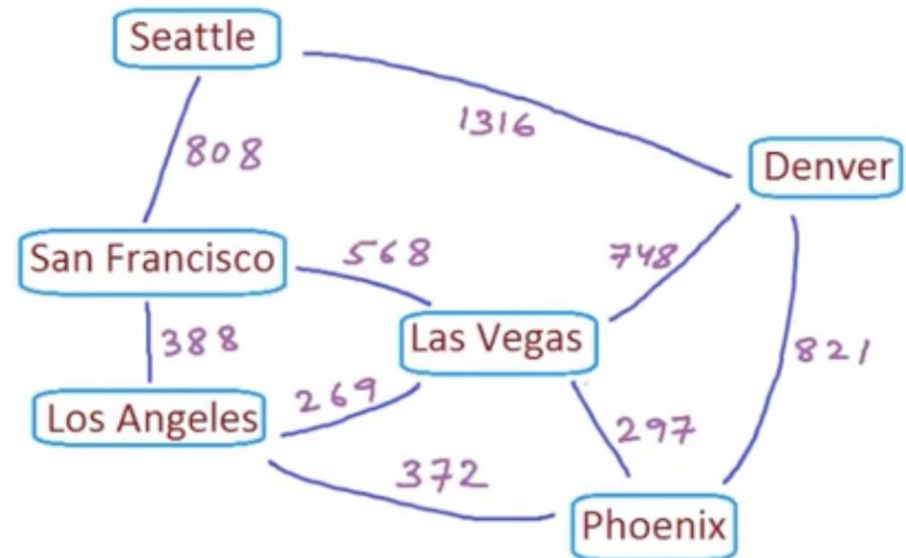


Vertex List

Seattle
San Francisco
Los Angeles
Las Vegas
Denver
Phoenix

Edge List

Seattle	San Francisco	808
Seattle	Denver	1316
San Francisco	Los Angeles	388
San Francisco	Las Vegas	568
Los Angeles	Las Vegas	269
Los Angeles	Phoenix	372
Las Vegas	Phoenix	297
Las Vegas	Denver	748
Phoenix	Denver	821



Intercity road-network



图的模板基类



```
const int maxWeight = .....;           //无穷大的值(=∞)
const int DefaultVertices = 30;         //最大顶点数(=n)
template <class T, class E>
class Graph {                             //图的类定义
protected:
    int maxVertices;                       //图中最大顶点数
    int numEdges;                          //当前边数
    int numVertices;                       //当前顶点数
    virtual int getVertexPos (T vertex);
    //给出顶点vertex在图中位置
public:
```



```
Graph (int sz = DefaultVertices);           //构造函数
~Graph();                                   //析构函数
bool GraphEmpty () const                    //判图空否
    { return numEdges == 0; }
int NumberOfVertices () { return numVertices; }
                                           //返回当前顶点数
int NumberOfEdges () { return numEdges; }
                                           //返回当前边数
virtual T getValue (int i);                 //取顶点 i 的值
virtual E getWeight (int v1, int v2);       //取边上权值
virtual int getFirstNeighbor (int v);
                                           //取顶点 v 的第一个邻接顶点
```



```
virtual int getNextNeighbor (int v, int w);  
    //取邻接顶点 w 的下一邻接顶点  
virtual bool insertVertex (const T vertex);  
    //插入一个顶点vertex  
virtual bool insertEdge (int v1, int v2, E cost);  
    //插入边(v1,v2), 权为cost  
virtual bool removeVertex (int v);  
    //删去顶点 v 和所有与相关联边  
virtual bool removeEdge (int v1, int v2);  
    //在图中删去边(v1,v2)  
};
```

所有虚函数都与具体存储表示相关。



Vertex List

0	A
1	B
2	C
3	D
4	E
5	F
6	G
7	H
	↓

Adjacency Matrix

	0	1	2	3	4	5	6	7
0	0	1	1	1	0	0	0	0
1	1	0	0	0	1	1	0	0
2	1	0	0	0	0	0	1	0
3	1	0	0	0	0	0	0	1
4	0	1	0	0	0	0	0	1
5	0	1	0	0	0	0	0	1
6	0	0	1	0	0	0	0	1
7	0	0	0	1	1	1	1	0

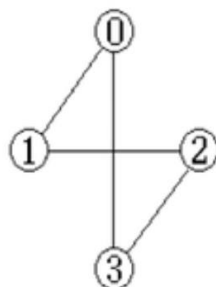


邻接矩阵 (Adjacency Matrix) 表示



- 在图的邻接矩阵表示中，有一个记录各个顶点信息的顶点表，还有一个表示各个顶点之间关系的邻接矩阵。
- 设图 $A = (V, E)$ 是一个有 n 个顶点的图, 图的邻接矩阵是一个二维数组 $A.Edge[n][n]$, 定义:

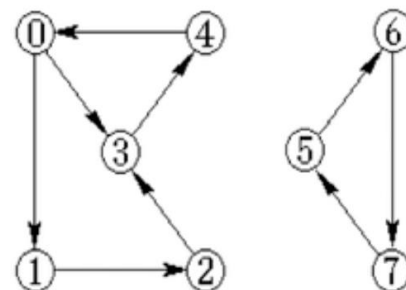
$$A.Edge[i][j] = \begin{cases} 1, & \text{如果 } \langle i, j \rangle \in E \text{ 或者 } (i, j) \in E \\ 0, & \text{否则} \end{cases}$$



G_8



G_3



G_7

$$A.Edge = \begin{pmatrix} \textcircled{0} & \textcircled{1} & \textcircled{2} & \textcircled{3} \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix} \begin{matrix} \textcircled{0} \\ \textcircled{1} \\ \textcircled{2} \\ \textcircled{3} \end{matrix}$$

$$A.Edge = \begin{pmatrix} \textcircled{0} & \textcircled{1} & \textcircled{2} \\ 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix} \begin{matrix} \textcircled{0} \\ \textcircled{1} \\ \textcircled{2} \end{matrix}$$

$$A.Edge = \begin{pmatrix} \textcircled{0} & \textcircled{1} & \textcircled{2} & \textcircled{3} & \textcircled{4} & \textcircled{5} & \textcircled{6} & \textcircled{7} \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ \hline 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix} \begin{matrix} \textcircled{0} \\ \textcircled{1} \\ \textcircled{2} \\ \textcircled{3} \\ \textcircled{4} \\ \textcircled{5} \\ \textcircled{6} \\ \textcircled{7} \end{matrix}$$

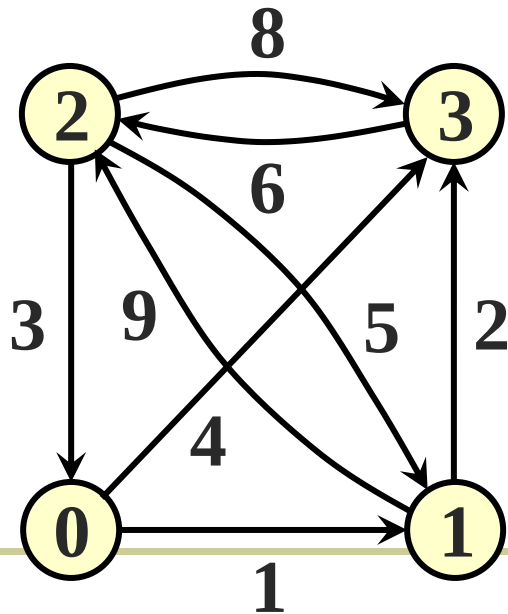


- 无向图的邻接矩阵是对称的;
- 有向图的邻接矩阵可能是不对称的。
- 在有向图中, 统计第 i 行 1 的个数可得顶点 i 的出度, 统计第 j 列 1 的个数可得顶点 j 的入度。
- 在无向图中, 统计第 i 行 (列) 1 的个数可得顶点 i 的度。



网络（带权图）的邻接矩阵

$$\mathbf{A.Edge}[i][j] = \begin{cases} \mathbf{W}(i, j), & \text{若 } i \neq j \text{ 且 } \langle i, j \rangle \in E \text{ 或 } (i, j) \in E \\ \infty, & \text{若 } i \neq j \text{ 且 } \langle i, j \rangle \notin E \text{ 或 } (i, j) \notin E \\ 0, & \text{若 } i == j \end{cases}$$



$$\mathbf{A.Edge} = \begin{bmatrix} 0 & 1 & \infty & 4 \\ \infty & 0 & 9 & 2 \\ 3 & 5 & 0 & 8 \\ \infty & \infty & 6 & 0 \end{bmatrix}$$



用邻接矩阵表示的图的类定义



```
template <class T, class E>
class Graphmtx : public Graph<T, E> {
friend istream& operator >> ( istream& in,  Graphmtx<T,
    E>& G);           //输入
friend ostream& operator << (ostream& out, Graphmtx<T,
    E>& G);           //输出
private:
    T *VerticesList;           //顶点表
    E **Edge;                 //邻接矩阵
    int getVertexPos (T vertex) {
        //给出顶点vertex在图中的位置
        for (int i = 0; i < numVertices; i++)
            if (VerticesList[i] == vertex) return i;
        return -1;
    };
};
```



public:

```
Graphmtx (int sz = DefaultVertices);
```

//构造函数

```
~Graphmtx ()
```

//析构函数

```
{ delete [ ]VerticesList;
```

```
for (i = 0; i < maxVertices; i++)
```

```
delete [ ] Edge [i];
```

```
delete [ ] Edge; }
```

```
T GetValue (int i) { //取顶点 i 的值, i 不合理返回0
```

```
return i >= 0 && i < numVertices ?
```

```
VerticesList[i] : NULL;
```

```
}
```

```
E GetWeight (int v1, int v2) { //取边(v1,v2)上权值
```

```
return v1 != -1 && v2 != -1 ? Edge[v1][v2] : 0;
```

```
}
```



```
int getFirstNeighbor (int v);  
                //取顶点 v 的第一个邻接顶点  
int getNextNeighbor (int v, int w);  
                //取 v 的邻接顶点 w 的下一邻接顶点  
bool insertVertex (const T vertex);  
                //插入顶点vertex  
bool insertEdge (int v1, int v2, E cost);  
                //插入边(v1, v2),权值为cost  
bool removeVertex (int v);  
                //删去顶点 v 和所有与它相关联的边  
bool removeEdge (int v1, int v2);  
                //在图中删去边(v1,v2)  
  
};
```




```
template <class T, class E>
```

```
Graphmtx<T, E>::Graphmtx (int sz) {    //构造函数
```

```
    maxVertices = sz;
```

```
    numVertices = 0; numEdges = 0;
```

```
    int i, j;
```

```
    VerticesList = new T[maxVertices]; //创建顶点表
```

```
    Edge = (E **) new E*[maxVertices];
```

```
    for (i = 0; i < maxVertices; i++)
```

```
        Edge[i] = new E[maxVertices]; //邻接矩阵
```

```
    for (i = 0; i < maxVertices; i++)    //矩阵初始化
```

```
        for (j = 0; j < maxVertices; j++)
```

```
            Edge[i][j] = (i == j) ? 0 : maxWeight;
```

```
};
```



```
template <class T, class E>
int Graphmtx<T, E>::getFirstNeighbor (int v) {
    //给出顶点位置为v的第一个邻接顶点的位置,
    //如果找不到, 则函数返回-1
    if (v != -1) {
        for (int col = 0; col < numVertices; col++)
            if (Edge[v][col] > 0 && Edge[v][col] < maxWeight)
                return col;
    }
    return -1;
};
```



```
template <class T, class E>
int Graphmtx<T, E>::getNextNeighbor (int v, int w) {
    //给出顶点 v 的某邻接顶点 w 的下一个邻接顶点
    if (v != -1 && w != -1) {
        for (int col = w+1; col < numVertices; col++)
            if (Edge[v][col] > 0 && Edge[v][col] < maxWeight)
                return col;
    }
    return -1;
};
```

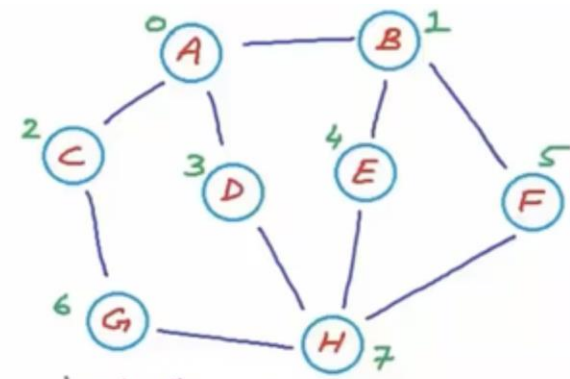
	0	1	2	3	4	5	6	7
0	0	1	1	1	0	0	0	0
1	1	0	0	0	1	1	0	0
2	1	0	0	0	0	0	1	0
3	1	0	0	0	0	0	0	1
4	0	1	0	0	0	0	0	1
5	0	1	0	0	0	0	0	1
6	0	0	1	0	0	0	0	1
7	0	0	0	1	1	1	1	0

`int A[8][8]`
 $\text{Space} = O(v^2)$

OR

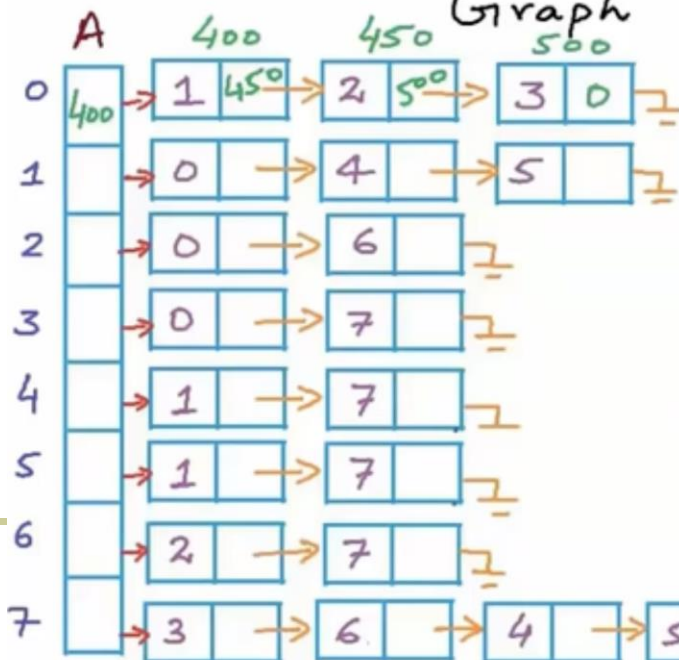
0	1	2	3	
1	0	4	5	
2	0	6		
3	0	7		
4	1	7		
5	1	7		
6	2	7		
7	3	6	4	5

$\text{Space} = O(e)$
 in most graphs,
 $e \ll v^2$



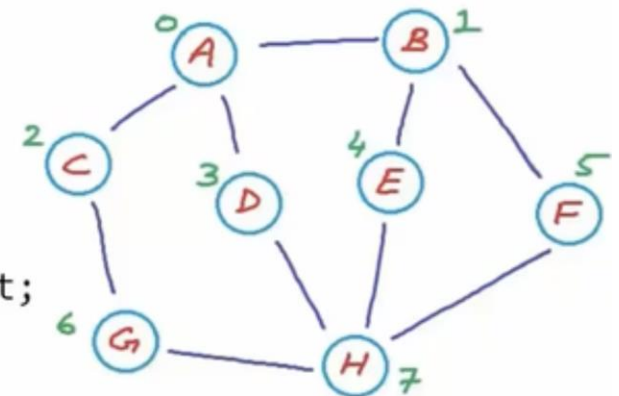
```
int *A[8];
A[0] = new int[3];
A[1] = new int[3];
A[2] = new int[2];
...
A[7] = new int[4];
```

Graph Representation



```
struct Node {
    int data;
    struct Node* next;
};

struct Node *A[8]
```



Space Complexity

$$= O(|E| + |V|)$$

Most graphs are
 sparse with

$$|E| \ll |V| * |V|$$



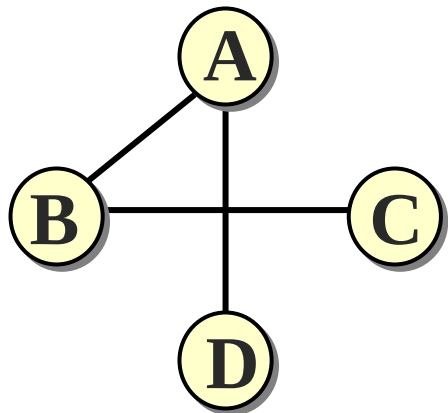
邻接表 (Adjacency List)



- 以邻接矩阵表示图，当边数远远小于 n^2 的时候，大量元素是maxWeight，会造成空间浪费。
- 邻接表是邻接矩阵的改进形式。为此需要把邻接矩阵的各行分别组织为一个**单链表**。
- 在邻接表中，同一个顶点发出的边链接在同一个**边链表**中，每一个链表结点代表一条边（**边结点**），结点中有另一顶点的标识 dest 和指针 link。对于带权图，边结点中还要保存该边的权值cost。
- **顶点表**的第 i 个顶点中保存该顶点的数据，以及它对应边链表的头指针adj。



无向图的邻接表

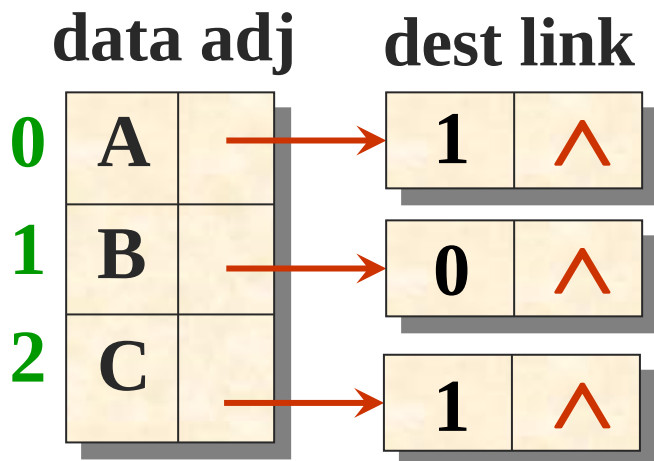
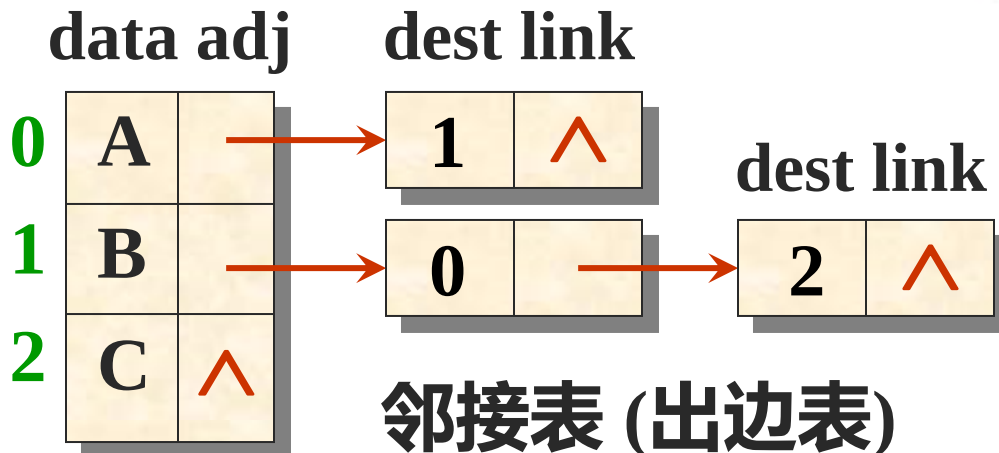


	data	adj	dest	link	dest	link
0	A	→	1	→	3	^
1	B	→	0	→	2	^
2	C	→	1	^		
3	D	→	0	^		

- 统计某顶点对应边链表中结点个数，可得该顶点的度。
- 某条边(v_i, v_j)在邻接表中有两个边结点，分别在第 i 个顶点和第 j 个顶点对应的边链表中。



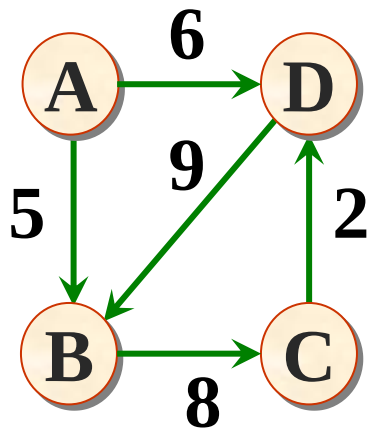
有向图的邻接表和逆邻接表



逆邻接表 (入边表)



网络(带权图)的邻接表



	data	adj	dest	cost	link
0	A	→	1	5	→ 3 6 ^
1	B	→	2	8	^
2	C	→	3	2	^
3	D	→	1	9	^

(顶点表)

(出边表)

- 统计出边表中结点个数, 得到该顶点的出度;
- 统计入边表中结点个数, 得到该顶点的入度。



用邻接表表示的图的类定义



```
template <class T, class E>
struct Edge {
    int dest;
    E cost;
    Edge<T, E> *link;
    Edge () {}
    Edge (int num, E c)
        : dest (num), cost (c), link (NULL) { }
    bool operator != (Edge<T, E>& R) const
        { return dest != R.dest ? true:false; } //判边等否
};
```

//边结点的定义
//边的另一顶点位置
//边上的权值
//下一条边链指针
//构造函数
//构造函数



```
template <class T, class E>
```

```
struct Vertex {
```

```
    T data;
```

```
    Edge<T, E> *adj;
```

```
};
```

//顶点的定义

//顶点的名字

//边链表的头指针

```
template <class T, class E>
```

```
class Graphlnk : public Graph<T, E> { //图的类定义
```

```
friend istream& operator >> (istream& in,
```

```
    Graphlnk<T, E>& G);           //输入
```

```
friend ostream& operator << (ostream& out,
```

```
    Graphlnk<T, E>& G);           //输出
```



private:

```
Vertex<T, E> *NodeTable;
```

```
//顶点表 (各边链表的头结点)
```

```
int getVertexPos (const T vertex) {
```

```
//给出顶点vertex在图中的位置
```

```
for (int i = 0; i < numVertices; i++)
```

```
    if (NodeTable[i].data == vertex) return i;
```

```
return -1;
```

```
}
```

public:

```
Graphlnk (int sz = DefaultVertices); //构造函数
```

```
~Graphlnk(); //析构函数
```



```
T getValue (int i) {                                //取顶点 i 的值
    return (i >= 0 && i < NumVertices) ?
        NodeTable[i].data : 0;
}

E getWeight (int v1, int v2);                        //取边(v1, v2)权值
bool insertVertex (const T& vertex);
bool removeVertex (int v);
bool insertEdge (int v1, int v2, E cost);
bool removeEdge (int v1, int v2);
int getFirstNeighbor (int v);
int getNextNeighbor (int v, int w);
};
```



```
template <class T, class E>
Graphlnk<T, E>::Graphlnk (int sz) {
    //构造函数： 建立一个空的邻接表
    maxVertices = sz;
    numVertices = 0; numEdges = 0;
    NodeTable = new Vertex<T, E>[maxVertices];
    //创建顶点表数组
    if (NodeTable == NULL)
        { cerr << "存储分配错！ " << endl; exit(1); }
    for (int i = 0; i < maxVertices; i++)
        NodeTable[i].adj = NULL;
};
```



```
template <class T, class E>
Graphlnk<T, E>::~~Graphlnk() {
    //析构函数： 删除一个邻接表
    for (int i = 0; i < maxVertices; i++ ) {
        Edge<T, E> *p = NodeTable[i].adj;
        while (p != NULL) {
            NodeTable[i].adj = p->link;
            delete p; p = NodeTable[i].adj;
        }
    }
    delete [ ]NodeTable; //删除顶点表数组
};
```



```
template <class T, class E>
int Graphlnk<T, E>::getFirstNeighbor (int v) {
//给出顶点位置为 v 的第一个邻接顶点的位置,
//如果找不到, 则函数返回-1
    if (v != -1) {                                //顶点v存在
        Edge<T, E> *p = NodeTable[v].adj;
        //对应边链表第一个边结点
        if (p != NULL) return p->dest;
        //存在, 返回第一个邻接顶点
    }
    return -1;    //第一个邻接顶点不存在
};
```



```
template <class T, class E>
int Graphlnk<T, E>::getNextNeighbor (int v, int w) {
//给出顶点v的邻接顶点w的下一个邻接顶点的位置,
//若没有下一个邻接顶点, 则函数返回-1
    if (v != -1) {                                //顶点v存在
        Edge<T, E> *p = NodeTable[v].adj;
        while (p != NULL && p->dest != w)
            p = p->link;
        if (p != NULL && p->link != NULL)
            return p->link->dest;    //返回下一个邻接顶点
    }
    return -1;                                //下一邻接顶点不存在
};
```




邻接矩阵与邻接表对比



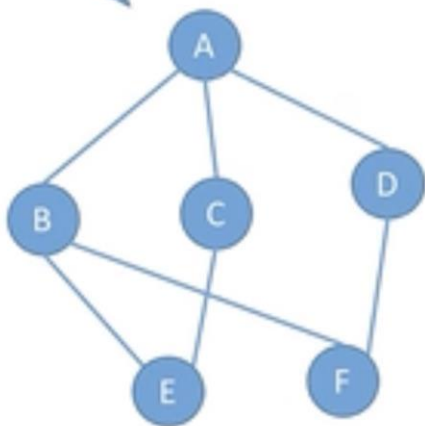
? Time requirements

■ 空间

■ 时间

Operation	Adjacency Matrix	Adjacency List
add edge	$O(1)$	$O(1)$
remove edge	$O(1)$	worst case: $O(\# \text{ successors})$, which can be as bad as $O(n)$
find edge	$O(1)$	worst case: $O(\# \text{ successors})$, which can be as bad as $O(n)$
list all successors	$O(n)$	worst case: $O(\# \text{ successors})$, which can be as bad as $O(n)$
initialize	$O(n^2)$	$O(n)$

无向图



邻接矩阵

	data		A	B	C	D	E	F
0	A	A	0	1	1	1	0	0
1	B	B	1	0	0	0	1	1
2	C	C	1	0	0	0	1	0
3	D	D	1	0	0	0	0	1
4	E	E	0	1	1	0	0	0
5	F	F	0	1	0	1	0	0

邻接表

	data	*first
0	A	→ 1 → 2 → 3 ^
1	B	→ 0 → 4 → 5 ^
2	C	→ 0 → 4 ^
3	D	→ 0 → 5 ^
4	E	→ 1 → 2 ^
5	F	→ 1 → 3 ^



邻接多重表 (Adjacency Multilist)



- 无向图的邻接表表示中，每条边被存储了两遍，既浪费了空间，又造成**为边做标记等有关边的处理**的不方便。
- 在邻接多重表中，每一条边只有一个边结点，为有关边的处理提供了方便。



无向图



◆ 边结点的结构

mark	vertex1	vertex2	path1	path2
------	---------	---------	-------	-------

- mark 是记录是否处理过的标记;
- vertex1和vertex2是该边两顶点位置。
- path1域是链接指针, 指向下一条依附顶点vertex1的边;
- path2 是指向下一条依附顶点vertex2的边链接指针。
- 需要时还可设置一个存放与该边相关的权值的域 cost。



无向图



◆ 顶点结点的结构

data	firstout
-------------	-----------------

- 存储顶点信息的结点表以顺序表方式组织, 每个顶点结点有两个数据成员: 其中, data 存放与该顶点相关的信息, firstout 是指向依附于该顶点的第一条边的指针。
- 在邻接多重表中, 所有依附同一个顶点的边都链接在同一个单链表中。

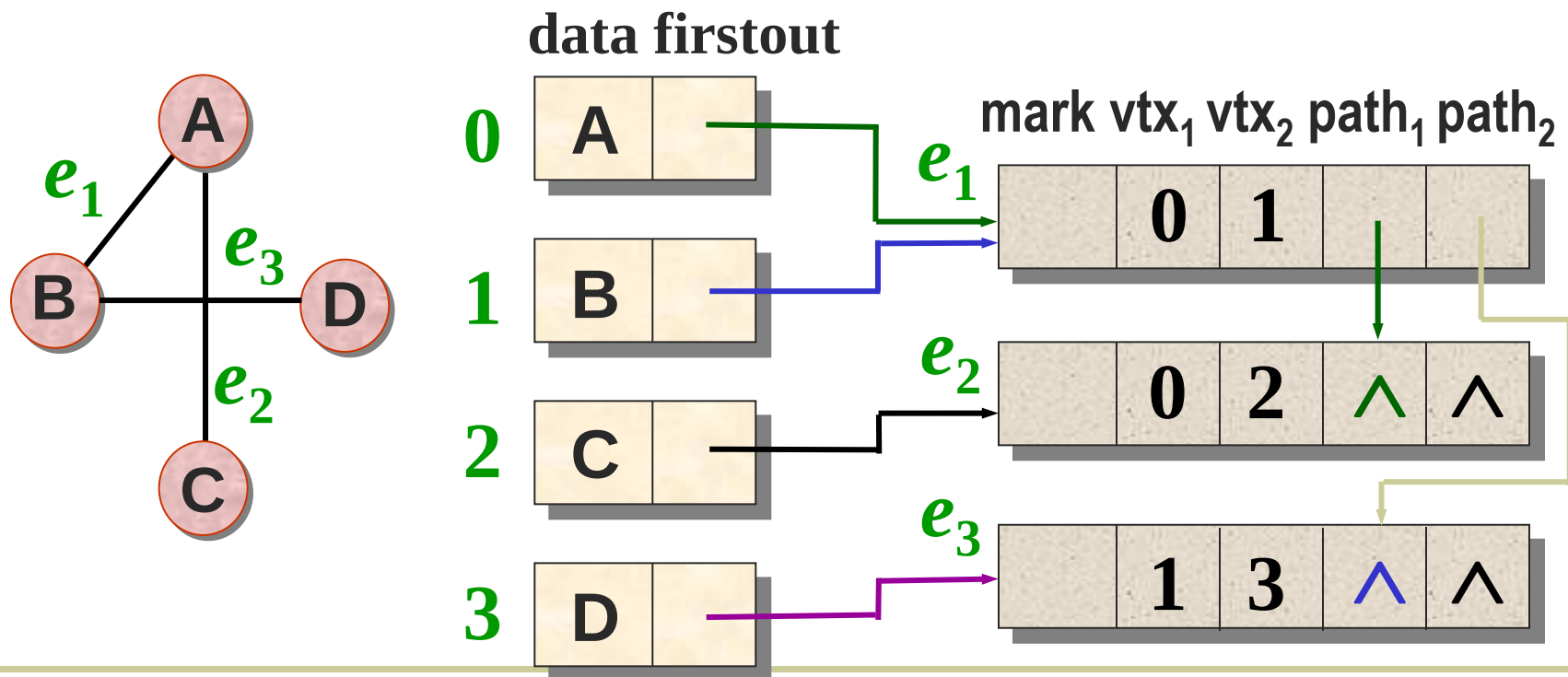


无向图



- 从顶点 i 出发, 可以循链找到所有依附于该顶点的边, 也可以找到它的所有邻接顶点。

邻接多重表的结构





有向图



- 在用邻接表表示有向图时,有时需要同时使用邻接表和逆邻接表。用有向图的邻接多重表(十字链表)可把两个表结合起来表示。
- ◆ 边结点的结构

mark	vertex1	vertex2	path1	path2
------	---------	---------	-------	-------

- 其中, mark是处理标记; vertex1和vertex2指明该有向边始顶点和终顶点的位置。path1是链接指针, 指向与该边有同一始顶点的下一条边(出边表); path2也是链接指针, 指向与该边有同一终顶点的下一条边(入边表)。需要时还可有权值域cost。



有向图



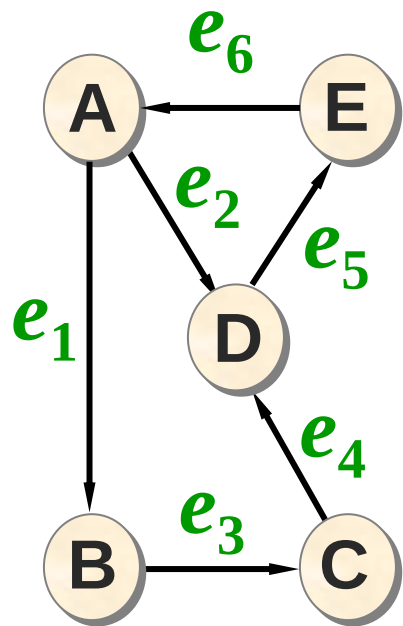
◆ 顶点结点的结构

data	firstin	firstout
------	---------	----------

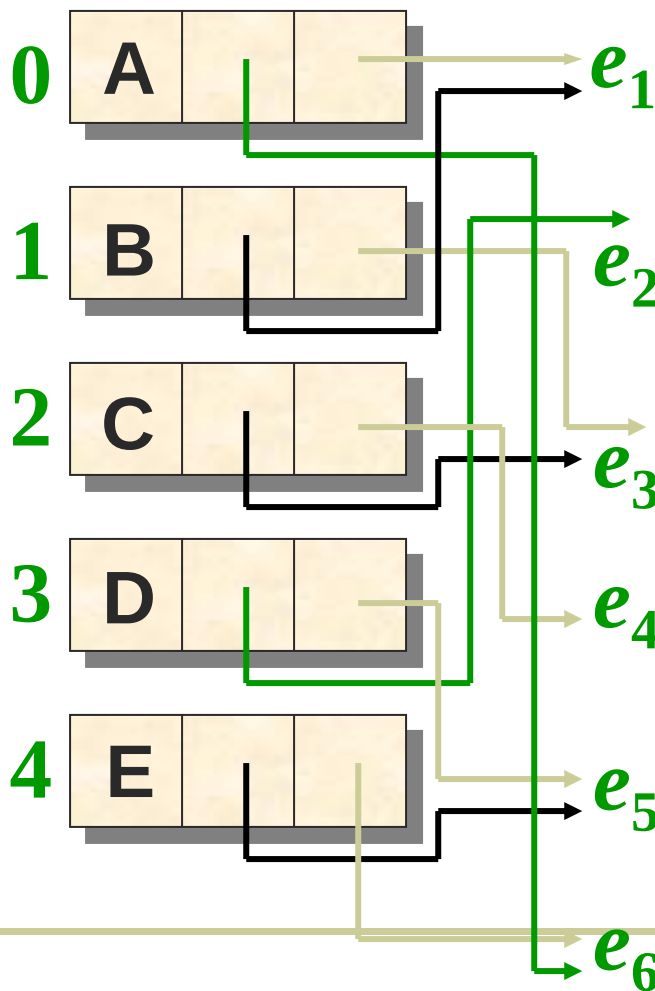
- 每个顶点有一个结点，它相当于出边表和入边表的表头结点：
 - 数据成员data存放与该顶点相关的信息
 - 指针firstout指向以该顶点为**始顶点**的出边表的第一条边
 - firstin 指向以该顶点为**终顶点**的入边表的第一条边



有向图



data firstin firstout



mark vtx₁ vtx₂ path₁ path₂

